COMP3702 Artificial Intelligence (Semester 2, 2025)
Assignment 1: Search in CHEESEHUNTER – **Report Template**

Name: Tsz Fung Kung

Student ID: 48996066

Student email: s4899606@student.uq.edu.au

Note: Please edit the name, student ID number and student email to reflect your identity and **do not modify the design or the layout in the assignment template**, including changing the paging.

Question 1 (Complete your full answer to Question 1 on the remainder page 1)

Dimension	Value	Justification
Planning horizon	Finite horizon	The game has a clear objective (getting the cheese after all traps/doors opened) and every action (walk/climb) has a fixed cost. As the result, the game will finish after a finite number of steps.
Representation	Factored	The state can be factored into player's position (row, column), trap/ lever status (activated/ not activated).
Computational limits	Bounded rationality	In theory, the whole state space can be enumerated, however the run time is too large (for level 5/6). In this assignment, the search method (UCS/A*) perform a bounded rationality by searching with lowest cost.
Learning	No learning	The agent (code) does not change after each run, the heuristics does not update from experience, so there is no online adaptation.
Sensing uncertainty	Fully observable	The gaming environment (structure, map, lever, goal location) are all given from the input file, so it is fully known to the agent.
Effect uncertainty	Deterministic	Every action (walk/climb/jump etc.) has a fully predictable outcome(which shown in the perform_action code); there is no randomness along the action.
Number of agents	Single agent	The is only one player (mouse) acts in the environment, no other opponents or collaborators.
Interactivity	Offline planning	The solver computes a complete plan using the search method before executing it. The environment do not change after each step the agent perform.

Question 2 (Complete your full answer to Question 2 on page 2)

1. Action Space

General definition: The action space is the finite set of control choices that is available to the agent in any state. Each action when applied to a state will invokes the environment's dynamics to produce a successor state, in this assignment, every action has a cost.

The possible actions are listed in the Game.Env.ACTIONS code: {"wl", "wr", "c", "d", "a", "j", "sl", "sr"} Each action has a positive cost (see the GameEnv. ACTION_COST code) There are also some codes use for checking the validity of action (see in part 3). When the action is put to the GameEnv.perform_action code, it will return True with the next state if it is valid, otherwise it will return false with a state.deepcopy() code which returns a new, independent GameState with the same configuration as before, which keeps the search algorithm safe and consistent.

2. State Space

General definition: The state space is the set of all configurations necessary and sufficient to predict outcomes of actions. Different states must encode all information that affects applicability and cost of actions and the goal test.

A state is the GameState tuple: s = (row, col, trap_status) implemented by the GameState class (game_state.py). row, col are integers indexing the grid, sizes are read from the testcase file by GameEnv.__init__. Trap_status is a tuple of 0/1 flags indicating for each trap whether its paired lever has it locked/open (1) or unlocked/openable (0). The length of this tuple equals the number of lever–trap pairs parsed from the level. GameState implements __eq__/__hash__, so it can be used as keys (e.g., in best_g dictionaries) by UCS/A* solver. The initial state is produced by GameEnv.get_init_state() and already contains the correct initial (row, col) and the initial trap_status tuple (all zeros unless a level specifies otherwise). The goal condition doesn't need extra state bits GameEnv.is_solved (state) decides it from the same fields.

3. Transition Function

General definition: The transition function T(s,a) returns the successor of applying action in state. In deterministic planning, it's a single next state (or failure); in stochastic planning, a distribution over next state.

In the game, there is a deterministic simulator: GameEnv.perform_action(state, action) implements the transition. It first calls check_valid_action (semantic preconditions by tile type, lever locations etc.). Also, it computes the candidate next (row,col) and the candidate next_trap_status. It calls check_collision to reject out-of-bounds, solid tiles, or stepping onto a closed drawbridge. If all checks pass, it returns (True, GameState(next_row, next_col, tuple(next_trap_status))); else (False, state.deepcopy()). Action costs are separate from T (see next section); the solver obtains them from GameEnv.ACTION_COST[action] and accumulates them into path cost g. Because transitions are deterministic and the world is static (aside from lever effects encoded in trap_status), the search can use standard graph search with duplicate detection (best_g).

4. Utility/ Cost Function

General definition: A utility (or cost) function assigns a scalar value to complete plans (or prefixes). In path-planning, we typically minimise the cumulative path cost: $\boldsymbol{Cost(\pi)=\sum_t c(a_t)}$ and the search algorithm (UCS/A*) finds a minimum-cost plan to a goal.

In this assignment, per-action costs are provided in GameEnv.ACTION_COST. The plan cost is the sum of action costs along the plan. Both UCS and A* in your solution.py implement this by maintaining g(s)g(s)g(s) (best known path cost to state s) and pushing priority keys ***g*** (UCS) or ***g+h*** (A*). Testcase files additionally supply target bands (min/max goals for cost, nodes, runtime), but they don't change optimality—UCS/A* still minimise the same additive cost. The goal test is independent of cost: GameEnv.is_solved(state) is true iff the player is at the exit and all traps are in the activated/locked state (i.e., every bit of trap_status is 1). Your solver stops when this condition is popped from the frontier with the optimal g (UCS) or first reached on-pop (A* with consistent heuristic).

Question 3 (Complete your full answer to Question 3 on page 3)

The result table:

Testcase	Algorithm	Visited / Reached (a)	Expanded (b)	Frontier at End (c)	Runtime (s) (d)
1	UCS	17	17	0	0.00011843
1	A*	17	17	0	0.00012153
2	UCS	75	70	5	0.00050283
2	A*	57	47	12	0.00036059
3	UCS	600	586	14	0.0042013
3	A*	487	480	8	0.0037032
4	UCS	4,567	4492	75	0.033547
4	A*	3,194	3,198	108	0.02536
5	UCS	818,184	801,403	16,912	7.3457
5	A*	420,712	417,486	12,923	5.4545
6	UCS	4,380,030	4,164,238	73,974	42.666
6	A*	2,146,916	2,139,575	42,860	31.011

The difference between A* and UCS algorithms:

UCS expands nodes in order of increasing path cost ***g(n)***. Which guarantees optimal solutions with positive costs, however as the search does not make use of any knowledge, the exploration is large and time consuming.

A* has a similar node expanding method with UCS, but with an advanced knowledge of computing the path, the heuristic. The path cost is known as ***f(n) = g(n) + h(n)***. By using the heuristic provided, the estimated cost to goal is being calculated. Which guide the search toward the goal effectively. As the result, A* search has a fewer expansion, smaller visited set and shorter runtimes than UCS while still optimal.

Comparison of numerical performance between algorithms and testcases:

Now using the table for comparison, for small map (testcase 1) the UCS and A* both behave similarly as the search space is tiny. The runtime difference is negligible while both expand almost the same number of nodes.

For the medium maps (testcase 2/3/4) we can start to see the different between using A* and UCS, the A* explores about 30% less (70> 47 (32%), 586 >480 (20%) and 4,492>3,198 (29%)) nodes than UCS and runtime noticeably lower, it should that the heuristic starts to guide the search efficiently. However, in testcase 4 the frontier sizes are slightly higher for A*, it may be due to heuristic prioritisation leaving more nodes waiting in the frontier.

For the much larger and complicated map (testcase 5/6), the differences between A* and UCS are dramatic. The expanded nodes have a about 50% reduction (801,403 > 417,486 (49%) and 4,164,238 > 2,139,575 (49%)) when using A* search, while the runtimes also dropped a lot, it shows that A* achieves the intended benefit with its heuristic: cutting search effort. Also, the frontier sizes of A*in bigger maps are smaller which shows its guidance focuses the search.

The heuristic is admissible and consistent which will be discussed in Q4, so the solutions remain optimal. In small maps, A* has no improvement as the maps is too small to be guided. In large maps, A*nearly halves expansions, this indicates the heuristic is start its effect on helping to calculate the nodes to visit. However, the number of reducing nodes cannot reach higher than 50% means that the heuristic is informative but not very tight. The trade-offs of the heuristic maybe it underestimates cost uniformly as many states share similar f. Which causes large plateaus of near-equal priority nodes so the expansions still high. The possible improvement of the heuristic is to implement a trap-aware distances or pattern database heuristics to further shrink expansions.

Question 4 (Complete your full answer to Question 4 on pages 4 and 5, and keep page 5 blank if you do not need it)

1. Heuristic Design

There is total 3 heuristic designed and used in the A* search. Which used to capture the most challenging aspects of the CheeseHunter environment: the asymmetric action costs, sprinting efficiency and lever order dependencies. The heuristic will be discussed below.

(a) Directed Movement Cost

Rationale: As the movement costs in ChessHunter are not uniform, walking(1block) costs 1 and sprinting (2blocks) cost only 1.9 (0.95 per cell). While climbing up costs 2 and dropping down cost only 0.5. We want to track down the movement by computing the direction-sensitive distances. For example, we want to make moving upward penalised more than moving downward. Which ensures that the heuristic reflects asymmetric movement dynamics.

Formula:

$$h_{\text{move}}(n) = dx \cdot c_{\text{horiz}} + \max(0, dy) \cdot c_{\text{up}} + \max(0, -dy) \cdot c_{\text{down}}$$

(b) Lever – Goal Ordering via MST

Rationale: The mouse must activate all levers before reaching the goal. We can calculate the minimum distance from the current state to any lever/goal, plus a Minimum Spanning Tree (MST) cost over all remaining levers and the goal. This lower-bounds the cost of visiting all mandatory locations without overestimating.

Formula:

$$h_{\text{lever}}(n) = \min_{p \in L \cup G} \big(\text{cost}(n, p)\big) + MST(L \cup G)$$

(c) Lever Activation Cost

Rationale: Each lever requires an explicit activate action (cost 1). We include a constant per remaining level to avoid underestimating.

Formula:

$$h_{\text{activate}}(n) = |L| \cdot c_{\text{activate}}$$

The combined Heuristic:

$$h(n) = h_{\text{move}}(n) + h_{\text{lever}}(n) + h_{\text{activate}}(n)$$

This ensures that movement costs, lever order and action overheads are all represented.

2. Admissibility

A heuristic h(n) is admissible if for all nodes n, $h(n) \leq C*(n)$, where $C*(n)$ is the true minimal cost from n to the goal. The heuristic used is admissible in the assignment. Here is the reason:

First, for the movement cost h_{move}, it uses minimum per-cell costs for each direction. For example, sprint cost per cell (0.95) is lower than walking (1),so we can assume the cheaper option is always available. This never overestimates.

Second, for the lever-goal ordering h_{lever}, it uses MST, which is a lower bound on the cost of visiting all inactivated levers and the goal.

Third, for the lever activation cost $h_{activate}$, it counts exactly one action per lever (by adding the activate cost into it), the true cost is least that much and not overestimates.

3. Computational Complexity and Accuracy

(a) Preprocessing & Caching

Step: During preprocessing, the heuristic compute pairwise directed costs between all levers and the goal and store them in a symmetric matrix.

Complexity: $O(L^2)$, where L is the number of levers. This is small in practice and amortized across all heuristic calls.

Justification: Avoids recomputing distances repeatedly, improving runtime efficiency.

(b) MST Computation

Step: For each heuristic evaluation, we compute MST over the set of inactivated levers + goal (Prim's algorithm).

Complexity: $O(k^2)$, where k is the number of remaining levers. With caching of previously computed masks, repeated states are constant time.

Accuracy: The MST gives a tighter lower bound than simple pairwise minimum distance, leading to fewer node expansions in A*.

(c) Trade-off : Accuracy vs. Complexity

Accuracy: By combining directional costs, MST ordering, and activation costs, the heuristic is much tighter than a naïve Manhattan distance.

Complexity: The extra overhead of MST is outweighed by reduced expansions (as confirmed in Question 3's results: A* nearly halved expansions vs UCS in large testcases).

Trade-off: For very small maps, UCS may be equally fast due to MST overhead, but in large maps, the heuristic pays off significantly.

4. Conclusion

The heuristic balances admissibility, efficiency, and accuracy by

1. captures asymmetric action costs and sprint efficiency.

2. accounts for lever order dependencies through MST.

3. ensures admissibility by always underestimating.

4. Precomputation and caching keep runtime overhead manageable.

As seen in the experimental results, this heuristic achieves the intended benefit of A* over UCS — fewer expanded nodes and faster runtime on large maps.

Appendix/References

References

Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

Poole, D., & Mackworth, A. (2023). Artificial Intelligence: Foundations of Computational Agents (3rd ed.). Cambridge University Press. Available at: https://artint.info

COMP3702 Lecture and Tutorial Materials (University of Queensland, Semester 2, 2025).

Generative AI Declaration

Tool	**Use**	**Prompt(s) / Task**	**Section(s)**	**Date**
ChatGPT (GPT-5)	Clarifying coding concepts and generating sample code for caching	"Explain how caching/memorization works and show a sample implementation in Python"	Solution.py (Heuristic Design, Measuring Distances, Sub-Problems, Memorization)	August 2025
ChatGPT (GPT-5)	Drafting and polishing written explanations for assignment report	"Help structure and polish my answer for Q2, Q3, and Q4, and check for the gramma mistake"	Assignment Report Q2–Q4	August 2025